Pract-1A Breadth First Search

```
import queue as Q
from RM import dict_gn
start='Arad'
goal='Bucharest'
result=''
def BFS(city, cityq, visitedq):
    global result
    if city==start:
        result=result+' '+city
    for eachcity in dict_gn[city].keys():
        if eachcity==goal:
            result=result+' '+eachcity
            return
        if eachcity not in cityq.queue and eachcity not in visitedq.queue:
            cityq.put(eachcity)
            result=result+' '+eachcity
    visitedq.put(city)
    BFS(cityq.get(),cityq,visitedq)
def main():
    cityq=Q.Queue()
    visitedq=Q.Queue()
    BFS(start, cityq, visitedq)
    print("BFS Traversal from",start,"to",goal,"is:")
    print(result)

main()
```

Pract-1B Iterative Depth First Search

```
import queue as Q
from RM import dict_gn
start='Arad'
goal='Bucharest'
result=''

def DLS(city, visitedstack, startlimit, endlimit):
    global result
    found=0
    result=result+city+' '
    visitedstack.append(city)
    if city==goal:
        return 1
    if startlimit==endlimit:
        return 0
    for eachcity in dict_gn[city].keys():
        if eachcity not in visitedstack:
            found=DLS(eachcity, visitedstack, startlimit+1, endlimit)
            if found:
                return found

def IDDFS(city, visitedstack, endlimit):
    global result
    for i in range(0,endlimit):
        print("Searching at Limit: ",i)
        found=DLS(city, visitedstack, 0, i)
        if found:
            print("Found")
            break
        else:
```

```
        print("Not Found!")
        print(result)
        print("-----")
        result=' '
        visitedstack=[]
def main():
   visitedstack=[]
   IDDFS(start, visitedstack, 9)
   print("IDDFS Traversal from ",start,"to",goal,"is:")
   print(result)
main()
```

Pract-2A A* Search algorithm

```
import queue as Q
from RM import dict_gn
from RM import dict_hn
start='Arad'
goal='Bucharest'
result=''
def get_fn(citystr):
   cities=citystr.split(",")
   hn=gn=0
   for ctr in range(0, len(cities)-1):
      gn=gn+dict_gn[cities[ctr]][cities[ctr+1]]
   hn=dict_hn[cities[len(cities)-1]]
   return(hn+gn)
def expand(cityq):
   global result
   tot, citystr, thiscity=cityq.get()
   if thiscity==goal:
```

```
        result=citystr+"::"+str(tot)
        return
    for cty in dict_gn[thiscity]:
        cityq.put((get_fn(citystr+","+cty),citystr+","+cty, cty))
    expand(cityq)
def main():
    cityq=Q.PriorityQueue()
    thiscity=start
    cityq.put((get_fn(start),start,thiscity))
    expand(cityq)
    print("The A* path with the total is:")
    print(result)
main()
```

Pract-2B Best First Search

```
import queue as Q
from RM import dict_gn
from RM import dict_hn

start='Arad'
goal='Bucharest'
result=''

def get_fn(citystr):
    cities=citystr.split(",")
    hn=gn=0
    for ctr in range(0, len(cities)-1):
        gn=gn+dict_gn[cities[ctr]][cities[ctr+1]]
    hn=dict_hn[cities[len(cities)-1]]
    return(hn+gn)
```

```
def printout(cityq):
    for i in range(0,cityq.qsize()):
        print(cityq.queue[i])

def expand(cityq):
    global result
    tot,citystr,thiscity=cityq.get()
    nexttot=999
    if not cityq.empty():
        nexttot,nextcitystr,nextthiscity=cityq.queue[0]
    if thiscity==goal and tot<nexttot:
        result=citystr+'::'+str(tot)
        return
    print("Expanded city----------------------",thiscity)
    print("Second best f(n)-----------------------",nexttot)
    print("\n")
    tempq=Q.PriorityQueue()
    for cty in dict_gn[thiscity]:
        tempq.put((get_fn(citystr+','+cty),citystr+','+cty,cty))
    for ctr in range(1,3):
        ctrtot,ctrcitystr,ctrthiscity=tempq.get()
        if ctrtot<nexttot:
            cityq.put((ctrtot,ctrcitystr,ctrthiscity))
        else:
            cityq.put((ctrtot,citystr,thiscity))
            break
    printout(cityq)
    expand(cityq)

def main():
    cityq=Q.PriorityQueue()
```

```
    thiscity=start
    cityq.put((999,"NA","NA"))
    cityq.put((get_fn(start),start,thiscity))
    expand(cityq)
    print(result)
main()
```

Pract-3 Decision Tree Learning

```
import pandas
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
data = pandas.read_csv('iris.csv')
x = data.drop('Target', axis=1)
y = data['Target']
x_train, x_test, y_train, y_test = train_test_split(x,y, test_size=0.2,random_state=42)
clf = DecisionTreeClassifier()
clf.fit(x_train, y_train)
y_pred = clf.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy:{accuracy:.2f}")
plt.figure(figsize=(12,8))
plot_tree(clf, filled=True,feature_names=x.columns, class_names=data['Target'].unique())
plt.title("Decision Tress Visualization")
plt.show()
```

Pract-4 Feed Forward Back Propagation Neural Network

```
import numpy as np


def sigmoid(x):
  return 1/(1+np.exp(-x))


def sigmoid_derivative(x):
  return x*(1-x)


class NeuralNetwork:
  def __init__(self, input_size, hidden_size, output_size):
    self.weights_input_hidden = np.random.uniform(-1, 1,(input_size, hidden_size))
    self.weights_hidden_output = np.random.uniform(-1, 1,(hidden_size, output_size))


  def forward(self, inputs):
    self.hidden_input = np.dot(inputs, self.weights_input_hidden)
    self.hidden_output = sigmoid(self.hidden_input)
    self.output_input = np.dot(self.hidden_output, self.weights_hidden_output)
    self.predicted_output = sigmoid(self.output_input)
    return self.predicted_output


  def backward(self, inputs, target, learning_rate):
    error = target - self.predicted_output
    delta_output = error*sigmoid_derivative(self.predicted_output)
    error_hidden = delta_output.dot(self.weights_hidden_output.T)
    delta_hidden = error_hidden*sigmoid_derivative(self.hidden_output)
    self.weights_hidden_output += np.outer(self.hidden_output, delta_output)*learning_rate
    self.weights_input_hidden += np.outer(inputs, delta_hidden)*learning_rate
  def train(self, training_data, targets, epochs, learning_rate):
```

```
        for epoch in range(epochs):
            for i in range(len(training_data)):
                inputs = training_data[i]
                target = targets[i]
                self.forward(inputs)
                self.backward(inputs, target, learning_rate)
    def predict(self, inputs):
        return self.forward(inputs)
training_data = np.array([[0,0],[0,1],[1,0],[1,1]])
targets = np.array([[0],[1],[1],[0]])
input_size = 2
hidden_size = 4
output_size = 1
learning_rate = 0.1
epochs = 10000
nn = NeuralNetwork(input_size, hidden_size, output_size)
nn.train(training_data, targets, epochs, learning_rate)
for i in range(len(training_data)):
    inputs = training_data[i]
    prediction = nn.predict(inputs)
    print(f"Input: {inputs}, Predicted Output: {prediction}")
```

Pract-5 Support Vector Machine

```
import pandas as pd
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = pd.read_csv('iris.csv')
x = data.drop('Target', axis=1)
```

```
y = data['Target']
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

svm_classifier = SVC(kernel='linear')
svm_classifier.fit(x_train, y_train)
y_pred = svm_classifier.predict(x_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

Pract-6 Adaboost algorithm to create ensemble

```
import pandas
from sklearn import model_selection
from sklearn.ensemble import AdaBoostClassifier
url = "C:/Users/Pradnya/OneDrive/ドキュメント/syllabus/AI/pima-indians-diabetes.data.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi','age', 'class']
dataframe = pandas.read_csv(url, names=names)
array = dataframe.values
X = array[:,0:8]
Y = array[:,8]
seed = 7
num_trees = 30
model = AdaBoostClassifier(n_estimators=num_trees, random_state=seed)
results = model_selection.cross_val_score(model, X, Y)
print(results.mean())
```

Pract-7 Naïve Bayes' Classifier

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
clf = GaussianNB()
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Pract-8 K-Nearest Neighbors(K-NN)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
df = pd.read_csv('Iris.csv')
X = df.drop(['ID', 'Target'], axis=1).values
y = df['Target'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
k = 3
clf = KNeighborsClassifier(n_neighbors=k)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

```
classification_accuracy = accuracy_score(y_test, y_pred)
print("Classification Accuracy:",classification_accuracy)
```

Pract-9 Association Rule Mining

```
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules
import pandas as pd
dataset = [
   ['milk', 'bread', 'nuts'],
   ['milk', 'bread'],
   ['milk', 'eggs', 'nuts'],
   ['milk', 'bread', 'eggs'],
   ['bread', 'nuts'],
   ]
df = pd.DataFrame(dataset)
df_encoded = pd.get_dummies(df, prefix='', prefix_sep='')

frequent_itemsets = apriori(df_encoded, min_support=0.5, use_colnames=True)
print("Frequent Itemsets:")
print(frequent_itemsets)
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.0)

print("\nAssociation Rules:")
print(rules)
```

Pract-10 TensorFlow Tools

```
import tensorflow as tf
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
```

```
from tensorflow.keras.preprocessing.sequence import pad_sequences
emails = [
   "Buy cheap watches1 free shipping!",
   "Meeting for lunch today?",
   "claim your prize! You've on $1,000,000!",
   "Important meeting at 3 PM.",
]
labels = [1, 0, 1, 0]
max_words = 1000
max_len = 50
tokenizer = Tokenizer(num_words=max_words, oov_token="<OOV>")
tokenizer.fit_on_texts(emails)
sequences = tokenizer.texts_to_sequences(emails)
X_padded = pad_sequences(sequences, maxlen=max_len, padding="post", truncating="post")

model = tf.keras.Sequential([
   tf.keras.layers.Embedding(input_dim=max_words, output_dim=16, input_length=max_len),
   tf.keras.layers.Flatten(),
   tf.keras.layers.Dense(16, activation='relu'),
   tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
training_data = np.array(X_padded)
training_labels = np.array(labels)

model.fit(training_data, training_labels, epochs=10)
file_path = "spam.txt"

with open(file_path, 'r', encoding = "utf-8")as file:
   sample_email_text = file.read()
```

```
sequences_sample = tokenizer.texts_to_sequences([sample_email_text])

sample_email_padded = pad_sequences(sequences_sample, maxlen = max_len, padding =
"post", truncating = "post")


prediction = model.predict(sample_email_padded)

threshold = 0.5

if prediction > threshold:

   print(f"Sample Email ('{file_path}'): SPAM")

else:

   print(f"Sample Email ('{file_path}'): NOT SPAM")
```